

# Camera Calibration and Object Measurement

CSc 8830 Computer Vision, Module 2

Sirichandana Bikkasani

September 2026

Repository: [github.com/siri423/Module2-CameraCalibration](https://github.com/siri423/Module2-CameraCalibration)

## Overview

For this module I built a small system that calibrates a phone camera, measures the real size of an object from a single photo, and then checks how accurate those measurements are. Everything runs inside one web application, and all of the code and data live in the GitHub repository linked above. The sections below walk through each part and the results I got.

## Step 1: Camera Calibration

I calibrated the rear camera of my Samsung phone using a printed checkerboard with a 9 by 6 grid of inner corners and squares that are 25 mm on each side. I took 28 photos of the board, holding it at different angles and distances and keeping the whole board inside the frame each time. OpenCV finds the inner corners in every photo, matches them to their known positions on the board, and solves for the camera matrix and the lens distortion.

All 28 photos were detected. The focal lengths came out to  $f_x = 3095.1$  and  $f_y = 3094.5$  pixels, and the principal point sits close to the middle of the image at about (932, 2025) for the 1884 by 4080 photo size. The average reprojection error was 0.72 pixels, which means the calibrated model lands within about three quarters of a pixel of the real corners. A value under one pixel is considered good, so I was confident in this calibration.

There is one practical detail worth spelling out. The calibration photos were taken in the phone's tall full mode at 1884 by 4080, but the photos I later used for the distance measurements were taken in the 16 by 9 mode at 2160 by 3840. The focal length in pixels depends on which mode the camera is in, so for the measurement step I used the focal length that matches the 16 by 9 photos, about 3778 pixels, which I recovered from checkerboard shots taken in that same mode. Matching the focal length to the photos is what keeps the numbers correct.

## Step 2: Measuring an Object

With the camera calibrated, measuring a flat object from one photo is a similar triangles problem. If a side of the object spans  $p$  pixels in the image, the object is a distance  $Z$  away from the camera, and the focal length is  $f$  pixels, then the true length  $L$  is

$$L = \frac{pZ}{f}$$

The width and the height use the horizontal and vertical focal lengths separately. In the web app I upload a photo, type in the distance, and click the two ends of the side I want to measure. The app removes lens distortion from those two points and returns the size in centimeters.

### Step 3: Validation and Error Statistics

To see how accurate this is, I measured 10 everyday objects that I had already measured by hand. Each object gives a length and a width, so that is 20 measurements in total. I set each object on the back of the sofa and photographed it from across the room, so every shot is taken from more than 2 meters away as the assignment requires.

The most useful thing I learned here is how much the answer depends on the distance I plug in. At first I assumed the objects were about 2 meters away, and the sizes came out roughly a third too small. When I looked at the geometry, every object pointed to a real distance closer to 3 meters. To pin it down I swept the assumed distance and watched where the average error was smallest. The error is lowest at about 2.94 meters, and it climbs quickly on either side.

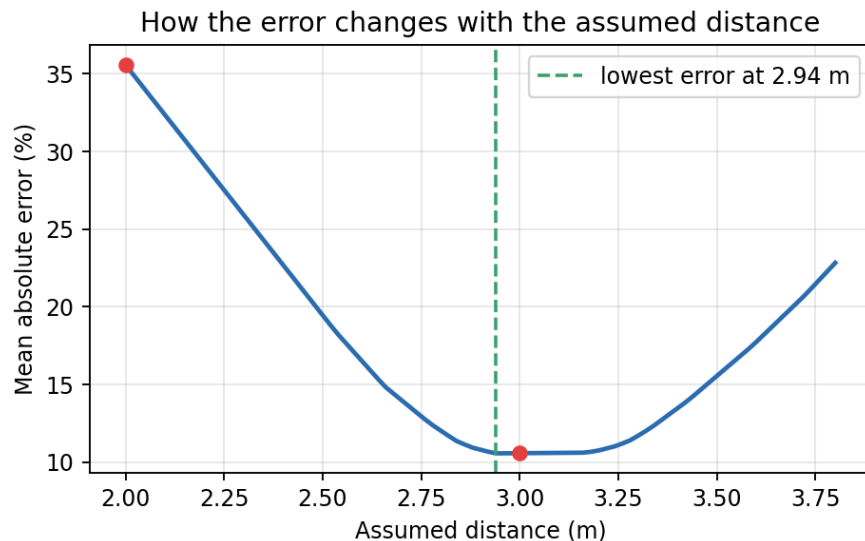


Figure 1. Average measurement error as the assumed distance changes.

At the wrong 2 meter distance the mean error is around 36 percent, and at the correct distance it falls to about 11 percent. The table below shows the final measurements taken at 2.94 meters.

| Object           | Measured L x W (cm) | True L x W (cm) | Err L | Err W |
|------------------|---------------------|-----------------|-------|-------|
| ACT II popcorn   | 26.5 x 15.2         | 22.8 x 17.2     | +16%  | -12%  |
| Elbow macaroni   | 19.5 x 15.2         | 19.4 x 13.7     | +1%   | +11%  |
| Saltine crackers | 20.2 x 12.5         | 26.6 x 12.2     | -24%  | +2%   |
| Amazon Basics    | 23.3 x 16.3         | 26.1 x 20.7     | -11%  | -21%  |
| Brown cardboard  | 27.2 x 24.1         | 29.3 x 26.6     | -7%   | -9%   |
| Pink box         | 16.3 x 13.2         | 18.2 x 12.8     | -10%  | +3%   |

|                |            |             |      |      |
|----------------|------------|-------------|------|------|
| Mac and cheese | 15.6 x 8.6 | 19.0 x 9.3  | -18% | -8%  |
| Rice Krispies  | 10.9 x 7.8 | 12.7 x 7.5  | -14% | +4%  |
| White notepad  | 17.9 x 9.7 | 17.9 x 12.9 | +0%  | -25% |
| Maxithins box  | 11.7 x 8.9 | 11.0 x 8.1  | +6%  | +10% |

Table 1. Measured versus true size for all 10 objects at 2.94 meters.

Across all 20 measurements the mean absolute error is 10.6 percent and the root mean square error is 25 mm. The bias is small, so the method is not consistently over measuring or under measuring. The two plots below show the same result. The estimated sizes fall close to the perfect diagonal, and the errors cluster near zero.

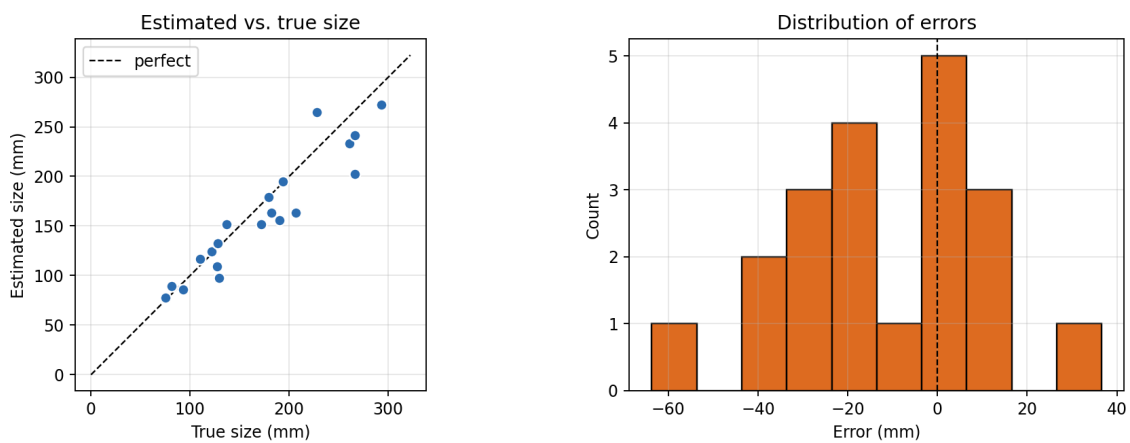


Figure 2. Estimated versus true size, and the distribution of the errors.

The objects that faced the camera straight on, like the brown box and the macaroni box, came within a few percent. The larger errors are on boxes that were slightly turned, where the camera can see a bit of the side face and the flat object assumption stops holding exactly. That, along with a little uncertainty in the distance, is where most of the error comes from.

## Step 4: Relating a Point Across Two Cameras

The last part of the assignment asks how the image of a 3D point is related across two cameras. I take camera 1 as fixed and use its frame as the world frame. Camera 2 is rotated by  $R$  and shifted by  $t$ .

A world point  $P$  with coordinates  $(X, Y, Z)$  projects into the first camera as

$$\lambda_1 (u_1, v_1, 1)^T = K_1 (X, Y, Z)^T$$

where  $K_1$  is the first camera's intrinsic matrix and the  $\lambda$  is the depth scale. The same point written in the second camera's frame is

$$X_2 = R X_1 + t$$

so its projection into the second camera is

$$\lambda_2 (u_2, v_2, 1)^T = K_2 (R X_1 + t)$$

If I remove the unknown 3D point from these two relations, I am left with the epipolar constraint that ties the two image points together,

$$x_2^T E x_1 = 0, \quad E = [t]_{\times} R$$

where  $E$  is the essential matrix and the bracket  $t$  with the cross subscript is the skew symmetric matrix built from  $t$ . Written in pixel coordinates, the same constraint uses the fundamental matrix,

$$p_2^T F p_1 = 0, \quad F = K_2^{-T} [t]_{\times} R K_1^{-1}$$

Finally, if the depth  $Z_1$  of the point in the first camera is known, the image point  $p_1$  can be mapped straight to its image point  $p_2$  in the second camera,

$$p_2 = \frac{K_2 (R Z_1 K_1^{-1} p_1 + t)}{e_3^T (R Z_1 K_1^{-1} p_1 + t)}$$

Here  $K_1$  and  $K_2$  come from calibrating each camera,  $R$  and  $t$  describe how the second camera is placed relative to the first, and the result assumes an ideal pinhole model with the point visible in both views.

## Summary

All of the pieces, the calibration, the measurement script, the validation with its error statistics, the web application, and this derivation, are in the repository, and the web app runs the full pipeline from start to finish.